Team ID	Discussion Time	Project Title	Final Grade
12	11:15 AM	Variant 3: Spreadsheet Formula Language	

#	Student Name	Student ID	Main Role / Contribution
1	رامي كامل عريان إسكندر	20230199	AST(Function Calls and Built-in functions)
2	حازم قرني محمد خليل	20230165	Scanner / Lexical Analysis
3	كريم محمد عبدالمغني محمد	20230416	Parser / Recursive Descent
4	مريم سعيد محمد	20230540	AST Core and Formula Structure
5	سلمى كمال بدر	20230256	AST Expressions and Operator Handling
6	تقى عبد العزيزمحمد	20230142	AST Values and Cell References

Variant-Specific Checks

Variant	Quick Checks
Spreadsheet	☐ cell references are tokenized correctly ☐ function calls parse correctly ☐ comma-separated arguments recurse correctly ☐ formula syntax is not space-dependent

Common Checklist

Area	Checklist	Quick Note
Language Scope	☐ Team can describe the language clearly ☐ Scope is specific and manageable ☐ Required features are implemented or realistically planned ☐ Valid and invalid sample inputs exist	
Scanner / Lexical Analysis	☐ Token categories are clearly defined ☐ Token vs. lexeme distinction is understood ☐ Keywords, identifiers, literals, operators, delimiters handled correctly ☐ Scanner output is visible independently ☐ Lexical errors are reported clearly	
Parser / Recursive Descent	☐ Grammar is suitable for recursive descent ☐ Parser logic reflects grammar nonterminals ☐ Lookahead handling is clear ☐ Valid inputs parse correctly ☐ Invalid inputs are rejected appropriately	
AST / Structural Output	☐ AST or structured output is present ☐ AST is clearly different from token output ☐ Node kinds are meaningful ☐ Team can explain the AST design	
Testing and Diagnostics	☐ Valid, invalid, and nontrivial nested cases are tested ☐ Syntax errors use useful messages ☐ System fails gracefully on malformed input	
Understanding and Explanation	☐ Team can explain grammar, scanner-parser interface, and parser flow ☐ Team can answer technical questions without reading from notes ☐ Project is not just an evaluator or UI demo with thin parsing	

Red Flags	Final Notes
☐ Confuses tokens and lexemes ☐ AST is really a parse trace or evaluator output ☐ Only simple happy-path examples shown	

Spreadsheet Formula Grammar (EBNF)

formula = START_EQUALS comparison ;

comparison = expression { (GREATER | GREATER_EQUAL | LESS | LESS_EQUAL | COMPARE_EQUALS) expression } ;

expression = term { (PLUS | MINUS) term } ;

term = factor { (STAR | SLASH) factor } ;

factor = [MINUS] factor | INT | range | function | LPAREN comparison RPAREN ;

range = CELL [COLON CELL] ;

function = IDENTIFIER LPAREN [arguments] RPAREN ;

arguments = comparison { COMMA comparison } ;

TOKEN TYPES

- **START_EQUALS**: The character = at the beginning of the formula.
- **COMPARE_EQUALS**: The comparison operator ==.
- **INT**: A numeric constant (e.g., 100).
- **CELL**: An individual cell reference (e.g., A1).
- **IDENTIFIER**: A function name (e.g., SUM, VLOOKUP).
- **PLUS, MINUS, STAR, SLASH**: Standard arithmetic operators +, -, *, /.
- **GREATER, LESS, GREATER_EQUAL, LESS_EQUAL**: Comparison operators >, <, >=, <=.
- **LPAREN, RPAREN**: Grouping and function call syntax (,).
- **COMMA**: Separator for function arguments ,.
- **COLON**: Range operator : (e.g., used in A1:B10).
- **EOF**: Special marker for the end of the input stream.
- **INVALID**: Token used by the lexer to flag unrecognized characters.